EXERCISE-1

Q1. Setup Postgres Install any latest stable version. Run using the terminal and create a database and table in that.

-To install postgres go to : https://www.postgresql.org/download/ and follow the steps mentioned there.

Create a database:

```
postgres=# CREATE DATABASE exercise;
CREATE DATABASE
postgres=# \l
postgres=# \l
                                                 List of databases
    Name     |  Owner   | Encoding | Locale Provider |   Collate   |    Ctype    | ICU Locale | ICU Rules |   Access privileges
-------------+----------+----------+-----------------+-------------+-------------+------------+-----------+-----------------------
 exercise    | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           |
 postgres    | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           |
 practice_db | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           |
 template0   | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           | =c/postgres          +
             |          |          |                 |             |             |            |           | postgres=CTc/postgres
 template1   | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |            |           | =c/postgres          +
             |          |          |                 |             |             |            |           | postgres=CTc/postgres
(5 rows)
```

A table in that database:

```
exercise=# CREATE TABLE students(
exercise(# student_id SERIAL PRIMARY KEY,
exercise(# name TEXT NOT NULL,
exercise(# email TEXT UNIQUE NOT NULL);
CREATE TABLE
```

```
exercise-# \d students
                                  Table "public.students"
   Column   |  Type   | Collation | Nullable |                    Default
------------+---------+-----------+----------+-----------------------------------------------
 student_id | integer |           | not null | nextval('students_student_id_seq'::regclass)
 name       | text    |           | not null |
 email      | text    |           | not null |
Indexes:
    "students_pkey" PRIMARY KEY, btree (student_id)
    "students_email_key" UNIQUE CONSTRAINT, btree (email)
```

Q2. Create a table having all the DataTypes in PostgreSQL

Query   Query History

```sql
CREATE TABLE datatype_table(
    id SERIAL PRIMARY KEY,
    name TEXT ,
    age INTEGER,
    salary NUMERIC(10,2),
    is_active BOOLEAN,
    created_at TIMESTAMP,
    profile JSON,
    bit_col BIT(8)
)
```

id	name	age	salary	is_active	created_at	profile	bit_col
[PK] integer	text	integer	numeric (10,2)	boolean	timestamp without time zone	json	bit

Q3. Create a table with primary key constraints

Query Query History

```sql
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    admission_date DATE NOT NULL
);
```

	student_id [PK] bigint	first_name text	last_name text	email text	admission_date date

Q4. Run all basic CRUD queries on the table

CRUD OPERATIONS:

Create:

Query Query History

```sql
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    admission_date DATE NOT NULL
);
```

Create (Insert):

Query Query History

```sql
INSERT INTO students (first_name, last_name, email, admission_date)
VALUES
('Harsh', 'Raj', 'harsh@email.com', '2026-02-01'),
('Aman', 'Mittal', 'aman@email.com', '2026-02-15');
```

Read (Select):

Query Query History

```
SELECT * FROM students
```

	student_id [PK] integer	first_name text	last_name text	email text	admission_date date
1	1	Harsh	Raj	harsh@email.co...	2026-02-01
2	2	Aman	Mittal	aman@email.co...	2026-02-15

Update:

```
UPDATE students
SET last_name = 'Pandey',
    admission_date = '2025-03-02'
WHERE student_id = 2;
```

Delete:

```
DELETE FROM students
WHERE student_id = 2;
```

Q5. Implement WHERE, GROUP BY, ORDER BY, HAVING in filters.

First creating and inserting values into a table to perform these tasks.

```sql
CREATE TABLE sales (
    sale_id SERIAL PRIMARY KEY,
    product_name TEXT NOT NULL,
    category TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    sale_date DATE NOT NULL
);
```

```sql
INSERT INTO sales (product_name, category, quantity, price, sale_date)
VALUES
('Laptop', 'Electronics', 2, 50000, '2024-01-10'),
('Mobile', 'Electronics', 5, 20000, '2024-01-12'),
('Shirt', 'Clothing', 10, 1500, '2024-01-15'),
('Jeans', 'Clothing', 4, 2500, '2024-01-18'),
('TV', 'Electronics', 1, 60000, '2024-01-20');
```

WHERE CLAUSE:

```sql
SELECT * FROM sales
WHERE category = 'Electronics';
```

	sale_id [PK] integer	product_name text	category text	quantity integer	price numeric (10,2)	sale_date date
1	1	Laptop	Electroni...	2	50000.00	2024-01-10
2	2	Mobile	Electroni...	5	20000.00	2024-01-12
3	5	TV	Electroni...	1	60000.00	2024-01-20

GROUP BY CLAUSE:

Query Query History

```sql
SELECT category, SUM(quantity) AS total_quantity
FROM sales
GROUP BY category;
```

	category text	total_quantity bigint
1	Electroni...	8
2	Clothing	14

HAVING CLAUSE:

Query Query History

```sql
SELECT category, SUM(quantity) AS total_quantity
FROM sales
GROUP BY category
HAVING SUM(quantity) > 9;
```

	category text	total_quantity bigint
1	Clothing	14

ORDER BY:

Query   Query History

```sql
SELECT *
FROM sales
ORDER BY price DESC
```

	sale_id [PK] integer	product_name text	category text	quantity integer	price numeric (10,2)	sale_date date
1	5	TV	Electroni...	1	60000.00	2024-01-20
2	2	Mobile	Electroni...	5	20000.00	2024-01-12
3	4	Jeans	Clothing	4	2500.00	2024-01-18
4	3	Shirt	Clothing	10	1500.00	2024-01-15